C# DSA
Mega Interview Booklet

150 Interview-Style Questions with Clean C# Solutions

Arrays & Hashing · Two Pointers · Sliding Window · Binary Search
Stack & Queue · Strings · Recursion & Backtracking · Dynamic Programming
Star Patterns · Basic Interview Classics · Number Theory · Sorting
Linked List · Trees · Greedy · Math & Bit Manipulation

Comprehensive Goto Reference · Pattern Cheatsheet · Complexity Guide
FAANG-Ready · LeetCode-Style · C# 10+ Idioms

2024 Edition — For C# Developers

Table of Contents

Chapter 00 — Comprehensive Goto Reference

Use this page as your starting point. Match the problem signal to the right pattern.

■ PROBLEM SIGNAL → PATTERN MAPPING

Array + find pair/triplet summing to X	Two Pointers (if sorted) / Hashing
Subarray or substring with constraint	Sliding Window
All combinations / permutations / subsets	Backtracking / Recursion
Min/Max of something + sorted input	Binary Search (on answer)
Matching brackets / nested structure	Stack
Shortest path in unweighted graph	BFS
Detect cycle / DFS order	DFS / Recursion
Overlapping subproblems + optimal structure	Dynamic Programming
Interval overlap / merge / schedule	Greedy + Sort by start
Frequency / count / lookup in O(1)	HashMap / HashSet
Next greater / smaller element	Monotonic Stack
Rank / kth element	Heap (PriorityQueue)
String edit distance / LCS / alignment	2D DP
Range sum query	Prefix Sum Array
Number of 1-bits / XOR tricks	Bit Manipulation

■ COMPLEXITY QUICK PICK

O(1)	Array index, HashMap get, Stack peek
O(log n)	Binary search, Balanced BST ops, Heap push/pop
O(n)	Single pass scan, Two pointers, Sliding window
O(n log n)	Sorting, Heap-based selection, Merge sort
O(n²)	Nested loops, Bubble/Insertion/Selection sort
O(2■)	Subsets / power set backtracking
O(n!)	Permutation backtracking
O(n·m)	2D DP, Grid BFS/DFS

■ C# DATA STRUCTURE CHOOSER

Need O(1) key lookup	Dictionary
Need O(1) membership test, no duplicates	HashSet
LIFO — undo, matching, DFS	Stack
FIFO — BFS, task queue	Queue
Min/Max element fast	PriorityQueue (.NET 6+)
Sorted unique keys	SortedSet / SortedDictionary
Frequent insert/delete at ends	LinkedList / Deque
Immutable sequence / pattern match	string (use StringBuilder to build)
Fixed-size random access	T[] array
Dynamic-size random access	List

■ DP SUBTYPE GUIDE

Count ways to reach N	1D DP — Climbing Stairs style
Min cost to complete N tasks	1D DP — Coin Change style
Max value with weight constraint	0/1 Knapsack — 2D DP
Longest subseq matching property	LIS / LCS — 1D or 2D DP
Optimal BST / Matrix chain	Interval DP
Grid path counting / min cost path	2D DP — dp[r][c]

■ INTERVIEW CHECKLIST (per problem)

1. Clarify	Edge cases? Negative nums? Empty input? Integer overflow?
2. Brute force	State O(n²) brute first, then optimise
3. Pattern	Identify: Two pointer? DP? Greedy? Hash?
4. Code	Write clean C#, meaningful variable names
5. Complexity	State Time and Space after coding
6. Test	Walk through at least 2 examples + edge case

■ KEY FORMULAS TO MEMORISE

Sum 1..n	n*(n+1)/2
Sum of squares 1..n	n*(n+1)*(2n+1)/6
Modular inverse	(a/b) % m = a * ModInverse(b,m) % m
nCr	n! / (r! * (n-r)!) — use Pascal's triangle for small
Max XOR	Trie-based O(n*32)
GCD (Euclidean)	gcd(a,b) = gcd(b, a%b)

Chapter 01 — Basic Interview Classics

These are the first questions every interviewer asks. Know them cold — they warm up the conversation.

Q1 Palindrome Check

■ Easy [String / Two Pointers]

Return true if a string reads the same forwards and backwards (ignore case & non-alphanumeric).

Clean the string first, then compare from both ends.

```
public bool IsPalindrome(string s) {
int l = 0, r = s.Length - 1;
while (l < r) {
while (l < r && !char.IsLetterOrDigit(s[l])) l++;
while (l < r && !char.IsLetterOrDigit(s[r])) r--;
if (char.ToLower(s[l]) != char.ToLower(s[r])) return false;
l++; r--;
}
return true;
}
```

■ O(n) | ■ O(1)

Q2 Factorial — Iterative & Recursive

■ Easy [Math / Recursion]

Compute n! both iteratively and recursively.

Iterative avoids stack overflow for large n. Recursive is elegant but limited.

```
// Iterative
public long FactIter(int n) {
long res = 1;
for (int i = 2; i <= n; i++) res *= i;
return res;
}
// Recursive
public long FactRec(int n) => n <= 1 ? 1 : n * FactRec(n - 1);
```

■ O(n) | ■ O(1) iter / O(n) recursive stack

Q3 Fibonacci — Iterative & Memoised

■ Easy [DP / Recursion]

Return nth Fibonacci number. F(0)=0, F(1)=1, F(n)=F(n-1)+F(n-2).

Iterative: track last two values. Memoised: cache with dictionary.

```
// O(n) iterative
public int Fib(int n) {
if (n <= 1) return n;
int a = 0, b = 1;
for (int i = 2; i <= n; i++) (a, b) = (b, a + b);
return b;
}
// Memoised recursive
var memo = new Dictionary<int,int>();
int FibMemo(int n) {
if (n <= 1) return n;
if (memo.TryGetValue(n, out int v)) return v;
return memo[n] = FibMemo(n-1) + FibMemo(n-2);
}
```

■ O(n) | ■ O(1) iter / O(n) memo

Q4 Reverse a String

■ Easy [String]

Reverse a string in-place using O(1) extra space.

Convert to char array, swap from both ends, reconstruct.

```
public string ReverseString(string s) {
char[] ch = s.ToCharArray();
int l = 0, r = ch.Length - 1;
while (l < r) { (ch[l], ch[r]) = (ch[r], ch[l]); l++; r--; }
return new string(ch);
}
```

■ O(n) | ■ O(n) for char array

Q5 Armstrong Number

■ Easy [Math]

A number is Armstrong if sum of its digits each raised to power (number of digits) equals the number. E.g. 153 = $1^3+5^3+3^3$.

Count digits, then sum each digit^count.

```
public bool IsArmstrong(int n) {
int digits = n.ToString().Length, sum = 0, temp = n;
while (temp > 0) {
int d = temp % 10;
sum += (int)Math.Pow(d, digits);
temp /= 10;
}
return sum == n;
}
```

■ O(d) | ■ O(1) where d = number of digits

Q6 Prime Number Check

■ Easy [Math]

Return true if n is prime. A prime has exactly two divisors: 1 and itself.

Only check up to √n. Skip evens after checking 2.

```
public bool IsPrime(int n) {
if (n < 2) return false;
if (n == 2) return true;
if (n % 2 == 0) return false;
for (int i = 3; (long)i*i <= n; i += 2)
if (n % i == 0) return false;
return true;
}
```

■ O(√n) | ■ O(1)

Q7 GCD and LCM

■ Easy [Math / Euclidean]

Find Greatest Common Divisor and Least Common Multiple of two numbers.

GCD via Euclidean algorithm. LCM = (a*b)/GCD(a,b).

```
public int GCD(int a, int b) => b == 0 ? a : GCD(b, a % b);
public long LCM(int a, int b) => (long)a / GCD(a, b) * b;
```

■ O(log(min(a,b))) | ■ O(log n) recursive stack

Q8 Power of Two

■ Easy [Bit Manipulation]

Given integer n, return true if it is a power of two.

A power of two has exactly one bit set. n & (n-1) == 0 clears that bit.

```
public bool IsPowerOfTwo(int n) => n > 0 && (n & (n - 1)) == 0;
```

■ O(1) | ■ O(1)

Q9 Sum of Digits

■ Easy [Math]

Repeatedly sum the digits of a number until a single digit. Return result and steps.

While n >= 10, sum its digits. Count iterations.

```
public (int result, int steps) DigitalRoot(int n) {
int steps = 0;
while (n >= 10) {
int s = 0; int t = n;
while (t > 0) { s += t % 10; t /= 10; }
n = s; steps++;
}
return (n, steps);
}
```

■ O(d * steps) | ■ O(1)

Q10 Count Vowels in a String

■ Easy [String]

Count the number of vowels (a,e,i,o,u — case insensitive) in a string.

Use a HashSet of vowels for O(1) lookup per character.

```
public int CountVowels(string s) {
var vowels = new HashSet<char>("aeiouAEIOU");
return s.Count(c => vowels.Contains(c));
}
```

■ O(n) | ■ O(1)

Chapter 02 — Star & Number Patterns

Pattern printing tests loop logic and indexing — a common warm-up in campus and junior developer interviews.

Q11 Right Triangle of Stars

■ Easy [Nested Loops]

Print a right-angled triangle of * with n rows. * ** ***

Outer loop = row, inner loop prints i stars.

```
public void RightTriangle(int n) {
for (int i = 1; i <= n; i++)
Console.WriteLine(new string('*', i));
}
```

■ O(n²) | ■ O(1)

Q12 Inverted Triangle

■ Easy [Nested Loops]

Print an inverted right triangle: n stars on first row, 1 on last.

```
public void InvertedTriangle(int n) {
for (int i = n; i >= 1; i--)
Console.WriteLine(new string('*', i));
}
```

■ O(n²) | ■ O(1)

Q13 Full Pyramid

■ Easy [Nested Loops]

Print a centred pyramid of stars with n rows.

Each row i has (n-i) leading spaces and (2*i-1) stars.

```
public void Pyramid(int n) {
for (int i = 1; i <= n; i++) {
Console.Write(new string(' ', n - i));
Console.WriteLine(new string('*', 2 * i - 1));
}
}
```

■ O(n²) | ■ O(1)

Q14 Diamond Pattern

■ Easy [Nested Loops]

Print a diamond: pyramid on top + inverted pyramid below.

```
public void Diamond(int n) {
for (int i = 1; i <= n; i++) {
Console.Write(new string(' ', n - i));
Console.WriteLine(new string('*', 2*i-1));
}
for (int i = n-1; i >= 1; i--) {
Console.Write(new string(' ', n - i));
Console.WriteLine(new string('*', 2*i-1));
}
}
```

■ O(n²) | ■ O(1)

Q15 Hollow Rectangle

■ Easy [Nested Loops]

Print a hollow rectangle of * with given rows and cols.

Only print * on border (first/last row or first/last column).

```
public void HollowRect(int rows, int cols) {
for (int i = 0; i < rows; i++) {
for (int j = 0; j < cols; j++) {
bool border = i==0||i==rows-1||j==0||j==cols-1;
Console.Write(border ? '*' : ' ');
}
Console.WriteLine();
}
}
```

■ O(n*m) | ■ O(1)

Q16 Pascal's Triangle

■ Medium [DP / Pattern]

Print first n rows of Pascal's Triangle. Each element = sum of two above.

Each row: start and end with 1. Middle = prev[j-1] + prev[j].

```
public void PascalTriangle(int n) {
var row = new List<long>();
for (int i = 0; i < n; i++) {
row.Insert(0, 1);
for (int j = 1; j < row.Count-1; j++)
row[j] = row[j] + row[j+1]; // built in place
// Simpler: build new row each time
}
}
// Cleaner version:
public IList<IList<int>> Generate(int n) {
var res = new List<IList<int>>();
for (int i = 0; i < n; i++) {
var row = new int[i+1];
row[0] = row[i] = 1;
for (int j=1; j<i; j++)
row[j] = res[i-1][j-1] + res[i-1][j];
res.Add(row);
}
return res;
}
```

■ O(n²) | ■ O(n)

Q17 Number Pyramid

■ Easy [Nested Loops]

Print a pyramid where each row i contains numbers 1..i. 1 12 123

```
public void NumberPyramid(int n) {
for (int i = 1; i <= n; i++) {
for (int j = 1; j <= i; j++) Console.Write(j);
Console.WriteLine();
}
}
```

■ O(n²) | ■ O(1)

Q18 Zigzag / Wave Pattern

■ Medium [Pattern]

Print characters of a string in zigzag pattern across numRows rows, then read row by row.

Track current row and direction. Reverse direction at top/bottom rows.

```
public string ZigzagConvert(string s, int numRows) {
if (numRows == 1) return s;
var sb = Enumerable.Range(0, numRows)
.Select(_ => new System.Text.StringBuilder()).ToArray();
int row = 0, dir = -1;
foreach (char c in s) {
sb[row].Append(c);
if (row == 0 || row == numRows-1) dir = -dir;
row += dir;
}
return string.Concat(sb.Select(b => b.ToString()));
}
```

■ O(n) | ■ O(n)

Q19 Butterfly Pattern

■ Easy [Nested Loops]

Print a butterfly pattern of * with n rows: * * ** ** *** ***

Top half: i stars, gap, i stars. Bottom half: mirror.

```
public void Butterfly(int n) {
for (int i = 1; i <= n; i++) {
Console.Write(new string('*', i));
Console.Write(new string(' ', 2*(n-i)));
Console.WriteLine(new string('*', i));
}
for (int i = n; i >= 1; i--) {
Console.Write(new string('*', i));
Console.Write(new string(' ', 2*(n-i)));
Console.WriteLine(new string('*', i));
}
}
```

■ O(n²) | ■ O(1)

Q20 Hourglass / Sandglass Pattern

■ Easy [Nested Loops]

Print an hourglass: inverted pyramid on top, pyramid on bottom, meeting at a point.

```
public void Hourglass(int n) {
for (int i = 0; i < n; i++) {
Console.Write(new string(' ', i));
Console.WriteLine(new string('*', 2*(n-i)-1));
}
for (int i = 1; i < n; i++) {
Console.Write(new string(' ', n-i-1));
Console.WriteLine(new string('*', 2*i+1));
}
}
```

■ $O(n^2)$ | ■ $O(1)$

Chapter 03 — Arrays & Hashing

Arrays + Dictionary = solving 80% of array problems in O(n). Master this duo.

Q21 Two Sum

■ Easy [Hashing]

Return indices of two numbers in nums that add to target.

```
public int[] TwoSum(int[] nums, int target) {
var map = new Dictionary<int,int>();
for (int i = 0; i < nums.Length; i++) {
int comp = target - nums[i];
if (map.ContainsKey(comp)) return new[]{map[comp], i};
map[nums[i]] = i;
}
return Array.Empty<int>();
}
```

■ O(n) | ■ O(n)

Q22 Contains Duplicate

■ Easy [HashSet]

Return true if any value appears at least twice in array.

```
public bool ContainsDuplicate(int[] nums) {
var seen = new HashSet<int>();
foreach (var n in nums) if (!seen.Add(n)) return true;
return false;
}
```

■ O(n) | ■ O(n)

Q23 Product of Array Except Self

■ Medium [Prefix/Suffix]

Return output[i] = product of all nums except nums[i]. No division. O(n) time.

Left pass: prefix products. Right pass: multiply suffix products.

```
public int[] ProductExceptSelf(int[] nums) {
int n = nums.Length;
int[] res = new int[n]; res[0] = 1;
for (int i=1; i<n; i++) res[i] = res[i-1]*nums[i-1];
int suffix = 1;
for (int i=n-1; i>=0; i--) { res[i]*=suffix; suffix*=nums[i]; }
return res;
}
```

■ O(n) | ■ O(1) extra

Q24 Top K Frequent Elements

■ Medium [Hashing + Bucket Sort]

Return k most frequent elements in O(n).

Bucket sort by frequency: bucket index = count.

```
public int[] TopKFrequent(int[] nums, int k) {
var freq = new Dictionary<int,int>();
foreach (var n in nums) freq[n] = freq.GetValueOrDefault(n,0)+1;
var buckets = new List<int>[nums.Length+1];
foreach (var (v,c) in freq) {
buckets[c] ??= new List<int>(); buckets[c].Add(v);
}
var res = new List<int>();
for (int i=buckets.Length-1; i>=1&&res.Count<k; i--)
if (buckets[i]!=null) res.AddRange(buckets[i]);
return res.Take(k).ToArray();
}
```

■ O(n) | ■ O(n)

Q25 Valid Anagram

■ Easy [Hashing]

Return true if t is an anagram of s.

```
public bool IsAnagram(string s, string t) {
if (s.Length != t.Length) return false;
var cnt = new int[26];
foreach (char c in s) cnt[c-'a']++;
foreach (char c in t) cnt[c-'a']--;
return cnt.All(x => x==0);
}
```

■ O(n) | ■ O(1)

Q26 Group Anagrams

■ Medium [Hashing]

Group strings so all anagrams are in the same list.

Sorted word = canonical key.

```
public IList<IList<string>> GroupAnagrams(string[] strs) {
var map = new Dictionary<string,List<string>>();
foreach (string s in strs) {
var key = string.Concat(s.OrderBy(c=>c));
if (!map.ContainsKey(key)) map[key]=new List<string>();
map[key].Add(s);
}
return map.Values.Cast<IList<string>>().ToList();
}
```

■ O(n·k·log k) | ■ O(n·k)

Q27 Longest Consecutive Sequence

■ Medium [HashSet]

Find length of longest sequence of consecutive integers. Must be O(n).

Only start counting when n-1 is NOT in set (n is sequence start).

```
public int LongestConsecutive(int[] nums) {
var set = new HashSet<int>(nums);
int best = 0;
foreach (int n in set) {
if (!set.Contains(n-1)) {
int cur=n, str=1;
while(set.Contains(cur+1)){cur++;str++;}
best = Math.Max(best,str);
}
}
return best;
}
```

■ O(n) | ■ O(n)

Q28 Find the Duplicate Number

■ Medium [Floyd's Cycle]

Array has n+1 ints in range [1,n]. Find the duplicate without extra space.

Treat array as linked list. Floyd's slow/fast pointer finds cycle.

```
public int FindDuplicate(int[] nums) {
int slow=nums[0], fast=nums[0];
do { slow=nums[slow]; fast=nums[nums[fast]]; } while(slow!=fast);
slow=nums[0];
while(slow!=fast){slow=nums[slow];fast=nums[fast];}
return slow;
}
```

■ O(n) | ■ O(1)

Q29 Majority Element

■ Easy [Boyer-Moore Voting]

Find element appearing more than n/2 times. Guaranteed to exist.

Candidate + count trick: cancel out pairs of different elements.

```
public int MajorityElement(int[] nums) {
int candidate=nums[0], count=1;
for (int i=1; i<nums.Length; i++) {
if (count==0) candidate=nums[i];
count += nums[i]==candidate ? 1 : -1;
}
return candidate;
}
```

■ O(n) | ■ O(1)

Q30 Rotate Array

■ Medium [Extra Space / Reversal]

Rotate array to the right by k steps.

Reverse whole array, then reverse first k, then reverse rest.

```
public void Rotate(int[] nums, int k) {
int n = nums.Length; k %= n;
void Rev(int l, int r) {
while(l<r){(nums[l],nums[r])=(nums[r],nums[l]);l++;r--;}
}
Rev(0,n-1); Rev(0,k-1); Rev(k,n-1);
}
```

■ O(n) | ■ O(1)

Q31 Maximum Subarray (Kadane's)

■ Medium [DP / Greedy]

Find the contiguous subarray with the largest sum.

curSum = max(num, curSum + num). Track max.

```
public int MaxSubArray(int[] nums) {
int cur = nums[0], best = nums[0];
for (int i=1; i<nums.Length; i++) {
cur = Math.Max(nums[i], cur+nums[i]);
best = Math.Max(best, cur);
}
return best;
}
```

■ O(n) | ■ O(1)

Q32 Merge Intervals

■ Medium [Sort + Greedy]

Merge all overlapping intervals.

Sort by start. If next start <= current end, merge; else append.

```
public int[][] Merge(int[][] intervals) {
Array.Sort(intervals, (a,b)=>a[0]-b[0]);
var res = new List<int[]>();
foreach (var iv in intervals) {
if (res.Count>0 && iv[0]<=res[^1][1])
res[^1][1]=Math.Max(res[^1][1],iv[1]);
else res.Add(iv);
}
return res.ToArray();
}
```

■ O(n log n) | ■ O(n)

Q33 Sort Colors (Dutch Flag)

■ Medium [Two Pointers]

Sort array with values 0,1,2 in-place in one pass.

Three pointers: lo (0s end), mid (current), hi (2s start).

```
public void SortColors(int[] nums) {
int lo=0, mid=0, hi=nums.Length-1;
while (mid<=hi) {
if (nums[mid]==0)(nums[lo],nums[mid])=(nums[mid],nums[lo++]);
else if (nums[mid]==2)(nums[mid],nums[hi])=(nums[hi],nums[mid--]);
if (nums[mid]!=2) mid++;
}
}
```

■ O(n) | ■ O(1)

Q34 Missing Number

■ Easy [Math / XOR]

Array 0..n with one missing. Find the missing number.

Expected sum = n*(n+1)/2. Subtract actual sum.

```
public int MissingNumber(int[] nums) {
int n = nums.Length;
return n*(n+1)/2 - nums.Sum();
}
```

■ O(n) | ■ O(1)

Q35 4Sum

■ Medium [Two Pointers]

Find all unique quadruplets that sum to target.

Sort. Fix two outer indices, use two pointers for inner. Skip duplicates.

```
public IList<IList<int>> FourSum(int[] nums, int target) {
Array.Sort(nums); int n=nums.Length;
var res = new List<IList<int>>();
for (int i=0; i<n-3; i++) {
if(i>0&&nums[i]==nums[i-1]) continue;
for (int j=i+1; j<n-2; j++) {
if(j>i+1&&nums[j]==nums[j-1]) continue;
int l=j+1,r=n-1;
while(l<r){
long s=(long)nums[i]+nums[j]+nums[l]+nums[r];
if(s==target){res.Add(new[]{nums[i],nums[j],nums[l],nums[r]});
while(l<r&&nums[l]==nums[l+1])l++;
while(l<r&&nums[r]==nums[r-1])r--;
l++;r--;}
else if(s<target)l++; else r--;
}
}
}
return res;
}
```

■ O(n³) | ■ O(1)

Chapter 04 — Two Pointers

Two pointers eliminate nested loops. One pointer from each end (or slow/fast) achieves O(n) over sorted structures.

Q36 3Sum

■ Medium [Two Pointers]

Find all unique triplets that sum to zero.

Sort. Fix index i, two-pointer [i+1, n-1]. Skip duplicates carefully.

```
public IList<IList<int>> ThreeSum(int[] nums) {
Array.Sort(nums); var res=new List<IList<int>>();
for(int i=0;i<nums.Length-2;i++){
if(i>0&&nums[i]==nums[i-1]) continue;
int l=i+1,r=nums.Length-1;
while(l<r){int s=nums[i]+nums[l]+nums[r];
if(s==0){res.Add(new[]{nums[i],nums[l],nums[r]});
while(l<r&&nums[l]==nums[l+1])l++;
while(l<r&&nums[r]==nums[r-1])r--;l++;r--;}
else if(s<0)l++; else r--;
}
}
return res;
}
```

■ O(n²) | ■ O(1)

Q37 Container With Most Water

■ Medium [Two Pointers]

Given heights array, maximise water between two lines.

Move the shorter pointer inward (only taller can improve area).

```
public int MaxArea(int[] h) {
int l=0,r=h.Length-1,best=0;
while(l<r){best=Math.Max(best,Math.Min(h[l],h[r])*(r-l));
if(h[l]<h[r])l++; else r--;}
return best;
}
```

■ O(n) | ■ O(1)

Q38 Trapping Rain Water

■ Hard [Two Pointers]

Compute total water trapped in elevation map.

water[i] = min(maxLeft, maxRight) - height[i]. Track both maxes with two pointers.

```
public int Trap(int[] h) {
int l=0,r=h.Length-1,ml=0,mr=0,w=0;
while(l<r){if(h[l]<=h[r]){ml=Math.Max(ml,h[l]);w+=ml-h[l];l++;}
else{mr=Math.Max(mr,h[r]);w+=mr-h[r];r--;}}
return w;
}
```

■ O(n) | ■ O(1)

Q39 Remove Duplicates from Sorted Array

■ Easy [Two Pointers]

Remove duplicates in-place from sorted array. Return new length.

Slow pointer tracks unique write position.

```
public int RemoveDuplicates(int[] nums) {
int slow=1;
for(int f=1;f<nums.Length;f++)
if(nums[f]!=nums[f-1]) nums[slow++]=nums[f];
return slow;
}
```

■ O(n) | ■ O(1)

Q40 Move Zeroes

■ Easy [Two Pointers]

Move all zeroes to end while maintaining relative order of non-zeroes.

Slow pointer = position for next non-zero. Swap when non-zero found.

```
public void MoveZeroes(int[] nums) {
int slow=0;
for(int f=0;f<nums.Length;f++)
if(nums[f]!=0)(nums[slow],nums[f])=(nums[f],nums[slow++]);
}
```

■ O(n) | ■ O(1)

Q41 Squares of a Sorted Array

■ Easy [Two Pointers]

Given sorted array (may have negatives), return sorted squares.

Two pointers from ends. Larger absolute value goes to end of result.

```
public int[] SortedSquares(int[] nums) {
int n=nums.Length,l=0,r=n-1,p=n-1;
int[] res=new int[n];
while(l<=r){
int ls=nums[l]*nums[l],rs=nums[r]*nums[r];
if(ls>rs){res[p--]=ls;l++;}else{res[p--]=rs;r--;}
}
return res;
}
```

■ O(n) | ■ O(n)

Q42 Boats to Save People

■ Medium [Greedy + Two Pointers]

Each boat holds 2 people with weight limit. Find minimum boats needed.

Sort. Pair lightest + heaviest if they fit; else heaviest goes alone.

```
public int NumRescueBoats(int[] people, int limit) {
Array.Sort(people);
int l=0,r=people.Length-1,boats=0;
while(l<=r){
if(people[l]+people[r]<=limit)l++;
r--; boats++;
}
return boats;
}
```

■ O(n log n) | ■ O(1)

Q43 Minimum Difference Between Closest Pair

■ Easy [Two Pointers]

Find the minimum absolute difference between any two elements in a sorted array.

Sort, then scan adjacent pairs.

```
public int MinDiff(int[] nums) {
Array.Sort(nums);
int min=int.MaxValue;
for(int i=1;i<nums.Length;i++)
min=Math.Min(min,nums[i]-nums[i-1]);
return min;
}
```

■ O(n log n) | ■ O(1)

Q44 Valid Palindrome II

■ Easy [Two Pointers]

Return true if string can become palindrome by deleting at most one character.

Two pointer. On mismatch, try skipping left OR right. Check both substrings.

```
public bool ValidPalindrome(string s) {
bool IsPalin(int l,int r){while(l<r){if(s[l]!=s[r])return false;l++;r--;}return true;}
int lo=0,hi=s.Length-1;
while(lo<hi){
if(s[lo]!=s[hi]) return IsPalin(lo+1,hi)||IsPalin(lo,hi-1);
lo++;hi--;
}
return true;
}
```

■ O(n) | ■ O(1)

Chapter 05 — Sliding Window

Template: expand right pointer, shrink left when constraint violated, track max/min window size.

Q45 Best Time to Buy and Sell Stock ■ Easy [Sliding Window]

Find max profit from one buy and one sell.

Track min price seen. Profit = current - minSoFar.

```
public int MaxProfit(int[] prices) {
int mn=int.MaxValue,best=0;
foreach(int p in prices){mn=Math.Min(mn,p);best=Math.Max(best,p-mn);}
return best;
}
```

■ O(n) | ■ O(1)

Q46 Longest Substring Without Repeating Characters ■ Medium [Sliding Window]

Find length of longest substring without repeating characters.

HashSet window. On duplicate, shrink from left.

```
public int LengthOfLongestSubstring(string s) {
var set=new HashSet<char>(); int l=0,mx=0;
for(int r=0;r<s.Length;r++){
while(set.Contains(s[r]))set.Remove(s[l++]);
set.Add(s[r]);mx=Math.Max(mx,r-l+1);
}
return mx;
}
```

■ O(n) | ■ O(k)

Q47 Minimum Window Substring

■ Hard [Sliding Window]

Find minimum window in s containing all chars of t.

Need/have counters. Expand right to satisfy, shrink left to minimise.

```
public string MinWindow(string s, string t) {
var need=new Dictionary<char,int>();
foreach(char c in t) need[c]=need.GetValueOrDefault(c,0)+1;
int have=0,needed=need.Count,l=0; int[] best={-1,0,0};
var win=new Dictionary<char,int>();
for(int r=0;r<s.Length;r++){
win[s[r]]=win.GetValueOrDefault(s[r],0)+1;
if(need.ContainsKey(s[r])&&win[s[r]]==need[s[r]])have++;
while(have==needed){
if(best[0]==-1||r-l+1<best[0])best=new[]{r-l+1,l,r};
win[s[l]]--;
if(need.ContainsKey(s[l])&&win[s[l]]<need[s[l]])have--;
l++;
}
}
return best[0]==-1?"":s.Substring(best[1],best[0]);
}
```

■ O(n+m) | ■ O(m)

Q48 Max Sum Subarray of Size K

■ Easy [Sliding Window (Fixed)]

Find maximum sum of any contiguous subarray of exactly size k.

Slide: subtract leftmost element, add new rightmost.

```
public int MaxSumSubarrayK(int[] nums, int k) {
int ws=nums.Take(k).Sum(),mx=ws;
for(int i=k;i<nums.Length;i++){
ws+=nums[i]-nums[i-k]; mx=Math.Max(mx,ws);
}
return mx;
}
```

■ O(n) | ■ O(1)

Q49 Longest Repeating Character Replacement

■ Medium [Sliding Window]

With at most k replacements, find longest substring with same letter.

(windowSize - maxFreq) > k → shrink. maxFreq only needs to grow.

```
public int CharacterReplacement(string s, int k) {
var cnt=new int[26]; int l=0,mf=0,res=0;
for(int r=0;r<s.Length;r++){
cnt[s[r]-'A']++;mf=Math.Max(mf,cnt[s[r]-'A']);
while((r-l+1)-mf>k)cnt[s[l++]-'A']--;
res=Math.Max(res,r-l+1);
}
return res;
}
```

■ O(n) | ■ O(1)

Q50 Permutation in String

■ Medium [Sliding Window + Hashing]

Return true if s2 contains a permutation of s1.

Fixed window of size s1.Length. Compare frequency maps.

```
public bool CheckInclusion(string s1, string s2) {
if(s1.Length>s2.Length) return false;
int[] n=new int[26],w=new int[26];
foreach(char c in s1) n[c-'a']++;
for(int i=0;i<s1.Length;i++) w[s2[i]-'a']++;
if(n.SequenceEqual(w)) return true;
for(int i=s1.Length;i<s2.Length;i++){
w[s2[i]-'a']++;w[s2[i-s1.Length]-'a']--;
if(n.SequenceEqual(w)) return true;
}
return false;
}
```

■ O(n) | ■ O(1)

Q51 Max Consecutive Ones III

■ Medium [Sliding Window]

Flip at most k zeros. Find max consecutive ones.

Track zeros in window. When zeros > k, shrink left.

```
public int LongestOnes(int[] nums, int k) {
int l=0,zeros=0,res=0;
for(int r=0;r<nums.Length;r++){
if(nums[r]==0) zeros++;
while(zeros>k) if(nums[l++]==0) zeros--;
res=Math.Max(res,r-l+1);
}
return res;
}
```

■ O(n) | ■ O(1)

Q52 Fruit Into Baskets

■ Medium [Sliding Window]

Pick max fruits using only 2 types (2 baskets). Return max fruits picked.

Sliding window with at most 2 distinct values. Shrink when 3rd type enters.

```
public int TotalFruit(int[] fruits) {
var basket=new Dictionary<int,int>(); int l=0,res=0;
for(int r=0;r<fruits.Length;r++){
basket[fruits[r]]=basket.GetValueOrDefault(fruits[r],0)+1;
while(basket.Count>2){
basket[fruits[l]]--;
if(basket[fruits[l]]==0)basket.Remove(fruits[l]);
l++;
}
res=Math.Max(res,r-l+1);
}
return res;
}
```

■ O(n) | ■ O(1)

Chapter 06 — Binary Search

Binary search applies to any monotonic decision space — not just sorted arrays. Ask: 'Can I binary search on the answer?'

Q53 Classic Binary Search

■ Easy [Binary Search]

Search sorted array for target. Return index or -1.

```
public int Search(int[] nums, int target) {
int lo=0,hi=nums.Length-1;
while(lo<=hi){int mid=lo+(hi-lo)/2;
if(nums[mid]==target)return mid;
else if(nums[mid]<target)lo=mid+1; else hi=mid-1;}
return -1;
}
```

■ O(log n) | ■ O(1)

Q54 Search in Rotated Sorted Array

■ Medium [Binary Search]

Search in array rotated at unknown pivot. No duplicates.

One half is always sorted. Determine which, then check if target lies in it.

```
public int SearchRotated(int[] nums, int target) {
int lo=0,hi=nums.Length-1;
while(lo<=hi){int mid=lo+(hi-lo)/2;
if(nums[mid]==target)return mid;
if(nums[lo]<=nums[mid]){
if(target>=nums[lo]&&target<nums[mid])hi=mid-1; else lo=mid+1;
}else{
if(target>nums[mid]&&target<=nums[hi])lo=mid+1; else hi=mid-1;
}
}
return -1;
}
```

■ O(log n) | ■ O(1)

Q55 Find Minimum in Rotated Sorted Array

■ Medium [Binary Search]

Find minimum element in rotated sorted array.

If mid > right, minimum is in right half; otherwise left half.

```
public int FindMin(int[] nums) {
int lo=0,hi=nums.Length-1;
while(lo<hi){int mid=lo+(hi-lo)/2;
if(nums[mid]>nums[hi])lo=mid+1; else hi=mid;}
return nums[lo];
}
```

■ O(log n) | ■ O(1)

Q56 Koko Eating Bananas

■ Medium [Binary Search on Answer]

Find minimum eating speed k to eat all piles within h hours.

Binary search k in [1, max(piles)]. Check feasibility each time.

```
public int MinEatingSpeed(int[] piles, int h) {
int lo=1,hi=piles.Max();
while(lo<hi){int mid=lo+(hi-lo)/2;
long hrs=piles.Sum(p=>(long)Math.Ceiling((double)p/mid));
if(hrs<=h)hi=mid; else lo=mid+1;}
return lo;
}
```

■ O(n log m) | ■ O(1)

Q57 First and Last Position of Element

■ Medium [Binary Search]

Find starting and ending position of target in sorted array.

Two binary searches: one for leftmost, one for rightmost occurrence.

```
public int[] SearchRange(int[] nums, int target) {
int First(){int lo=0,hi=nums.Length-1,res=-1;
while(lo<=hi){int m=lo+(hi-lo)/2;
if(nums[m]==target){res=m;hi=m-1;}
else if(nums[m]<target)lo=m+1; else hi=m-1;}return res;}
int Last(){int lo=0,hi=nums.Length-1,res=-1;
while(lo<=hi){int m=lo+(hi-lo)/2;
if(nums[m]==target){res=m;lo=m+1;}
else if(nums[m]<target)lo=m+1; else hi=m-1;}return res;}
return new[]{First(),Last()};
}
```

■ O(log n) | ■ O(1)

Q58 Search a 2D Matrix

■ Medium [Binary Search]

Matrix rows sorted, first element of each row > last of previous. Search target.

Treat as 1D sorted array. Index mapping: row=mid/cols, col=mid%cols.

```
public bool SearchMatrix(int[][] m, int target) {
int rows=m.Length,cols=m[0].Length,lo=0,hi=rows*cols-1;
while(lo<=hi){int mid=lo+(hi-lo)/2;
int v=m[mid/cols][mid%cols];
if(v==target)return true;
else if(v<target)lo=mid+1; else hi=mid-1;}
return false;
}
```

■ O(log(n*m)) | ■ O(1)

Q59 Median of Two Sorted Arrays

■ Hard [Binary Search]

Find median of two sorted arrays. Must run in O(log(m+n)).

Binary search partition on smaller array. Ensure left halves correctly partitioned.

```
public double FindMedianSortedArrays(int[] A, int[] B) {
if(A.Length>B.Length)(A,B)=(B,A);
int m=A.Length,n=B.Length,lo=0,hi=m;
while(lo<=hi){
int i=lo+(hi-lo)/2, j=(m+n+1)/2-i;
int aL=i>0?A[i-1]:int.MinValue, aR=i<m?A[i]:int.MaxValue;
int bL=j>0?B[j-1]:int.MinValue, bR=j<n?B[j]:int.MaxValue;
if(aL<=bR&&bL<=aR){
int maxL=Math.Max(aL,bL);
if((m+n)%2==1)return maxL;
return(maxL+Math.Min(aR,bR))/2.0;
}else if(aL>bR)hi=i-1; else lo=i+1;
}
return 0;
}
```

■ O(log(min(m,n))) | ■ O(1)

Chapter 07 — Stack & Queue

Stack (LIFO) solves matching, nesting, and monotonic problems. Use Stack in C#.

Q60 Valid Parentheses

■ Easy [Stack]

Check if string of brackets is valid.

Push on open. On close, check top matches.

```
public bool IsValid(string s) {
var st=new Stack<char>();
foreach(char c in s){
if(c=='('||c=='{'||c=='['){st.Push(c);continue;}
if(st.Count==0)return false;
char t=st.Pop();
if(c==')'&&t!='('||c=='}'&&t!='{'||c==']'&&t!='[')return false;
}
return st.Count==0;
}
```

■ O(n) | ■ O(n)

Q61 Daily Temperatures

■ Medium [Monotonic Stack]

For each day, how many days until a warmer temperature?

Monotonic decreasing stack of indices. When warmer found, pop and fill answer.

```
public int[] DailyTemperatures(int[] t) {
int n=t.Length; int[] res=new int[n];
var st=new Stack<int>();
for(int i=0;i<n;i++){
while(st.Count>0&&t[i]>t[st.Peek()]){int j=st.Pop();res[j]=i-j;}
st.Push(i);
}
return res;
}
```

■ O(n) | ■ O(n)

Q62 Min Stack

■ Medium [Stack Design]

Stack supporting push/pop/top and getMin in O(1).

Parallel minStack stores current minimum at each level.

```
public class MinStack {
Stack<int> s=new(),ms=new();
public void Push(int v){s.Push(v);
ms.Push(ms.Count==0?v:Math.Min(v,ms.Peek()));}
public void Pop(){s.Pop();ms.Pop();}
public int Top()=>s.Peek();
public int GetMin()=>ms.Peek();
}
```

■ O(1) all | ■ O(n)

Q63 Evaluate Reverse Polish Notation

■ Medium [Stack]

Evaluate an arithmetic expression in Reverse Polish Notation.

Push numbers. On operator, pop two, compute, push result.

```
public int EvalRPN(string[] tokens) {
var st=new Stack<long>();
foreach(var t in tokens){
if(long.TryParse(t,out long n)){st.Push(n);continue;}
long b=st.Pop(),a=st.Pop();
st.Push(t=="+"?a+b:t=="-"?a-b:t=="*"?a*b:a/b);
}
return(int)st.Pop();
}
```

■ O(n) | ■ O(n)

Q64 Next Greater Element

■ Medium [Monotonic Stack]

For each element in nums1 find next greater element in nums2. Return -1 if none.

Build next-greater map for nums2 using monotonic stack, then look up nums1.

```
public int[] NextGreaterElement(int[] nums1, int[] nums2) {
var map=new Dictionary<int,int>(); var st=new Stack<int>();
foreach(int n in nums2){
while(st.Count>0&&n>st.Peek())map[st.Pop()]=n;
st.Push(n);
}
return nums1.Select(n=>map.GetValueOrDefault(n,-1)).ToArray();
}
```

■ O(n+m) | ■ O(n)

Q65 Decode String

■ Medium [Stack]

Decode string like '3[a2[bc]]' → 'abcbcabcbcabcbc'.

Stack for counts and built strings. On ']', pop and repeat current segment.

```
public string DecodeString(string s) {
var cntStack=new Stack<int>(); var strStack=new Stack<string>();
string cur=""; int k=0;
foreach(char c in s){
if(char.IsDigit(c))k=k*10+(c-'0');
else if(c=='['){cntStack.Push(k);strStack.Push(cur);cur="";k=0;}
else if(c==']'){int rep=cntStack.Pop();
cur=strStack.Pop()+string.Concat(Enumerable.Repeat(cur,rep));}
else cur+=c;
}
return cur;
}
```

■ O(n·k) | ■ O(n)

Q66 Largest Rectangle in Histogram

■ Hard [Monotonic Stack]

Find the largest rectangle area in histogram.

Monotonic increasing stack. On smaller bar, pop and compute area.

```
public int LargestRectangleArea(int[] heights) {
var st=new Stack<int>(); int mx=0;
int[] h=heights.Append(0).ToArray();
for(int i=0;i<h.Length;i++){
while(st.Count>0&&h[i]<h[st.Peek()]){
int ht=h[st.Pop()];
int w=st.Count==0?i:i-st.Peek()-1;
mx=Math.Max(mx,ht*w);
}
st.Push(i);
}
return mx;
}
```

■ O(n) | ■ O(n)

Q67 Implement Queue Using Two Stacks

■ Easy [Stack / Design]

Implement FIFO queue using only two stacks.

Lazy transfer: move s1→s2 only when s2 is empty on peek/pop.

```
public class MyQueue {
Stack<int> s1=new(),s2=new();
public void Push(int x)=>s1.Push(x);
void Transfer(){if(s2.Count==0)while(s1.Count>0)s2.Push(s1.Pop());}
public int Pop(){Transfer();return s2.Pop();}
public int Peek(){Transfer();return s2.Peek();}
public bool Empty()=>s1.Count==0&&s2.Count==0;
}
```

■ Push O(1), Pop/Peek O(1) amortised | ■ O(n)

Chapter 08 — Strings

Use StringBuilder for O(n) construction. Strings are immutable in C# — concatenation in a loop is O(n²).

Q68 Reverse Words in a String

■ Medium [String]

Reverse the order of words. Remove extra spaces.

```
public string ReverseWords(string s) {
var w=s.Split(' ',StringSplitOptions.RemoveEmptyEntries);
Array.Reverse(w); return string.Join(' ',w);
}
```

■ O(n) | ■ O(n)

Q69 Longest Common Prefix

■ Easy [String]

Find longest common prefix among array of strings.

Trim prefix until all strings start with it.

```
public string LongestCommonPrefix(string[] strs) {
string pre=strs[0];
for(int i=1;i<strs.Length;i++)
while(!strs[i].StartsWith(pre)) pre=pre[..^1];
return pre;
}
```

■ O(n·m) | ■ O(1)

Q70 Roman to Integer

■ Easy [String + Hashing]

Convert Roman numeral to integer.

If current value < next value, subtract it; otherwise add.

```
public int RomanToInt(string s) {
var m=new Dictionary<char,int>{{'I',1},{'V',5},{'X',10},
{'L',50},{'C',100},{'D',500},{'M',1000}};
int t=0;
for(int i=0;i<s.Length;i++)
t+=i+1<s.Length&&m[s[i]]<m[s[i+1]]?-m[s[i]]:m[s[i]];
return t;
}
```

■ O(n) | ■ O(1)

Q71 Longest Palindromic Substring

■ Medium [Expand Around Center]

Find the longest palindromic substring in s.

For each center (or pair of centers), expand outward while characters match.

```
public string LongestPalindrome(string s) {
int start=0,maxLen=1;
void Expand(int l,int r){
while(l>=0&&r<s.Length&&s[l]==s[r]){
if(r-l+1>maxLen){start=l;maxLen=r-l+1;}
l--;r++;}}
for(int i=0;i<s.Length;i++){Expand(i,i);Expand(i,i+1);}
return s.Substring(start,maxLen);
}
```

■ O(n²) | ■ O(1)

Q72 Word Break

■ Medium [DP + Hashing]

Return true if s can be segmented into words from the dictionary.

dp[i] = true if s[0..i-1] can be segmented. For each j < i, check dp[j] + s[j..i].

```
public bool WordBreak(string s, IList<string> wordDict) {
var set=new HashSet<string>(wordDict);
bool[] dp=new bool[s.Length+1]; dp[0]=true;
for(int i=1;i<=s.Length;i++)
for(int j=0;j<i;j++)
if(dp[j]&&set.Contains(s.Substring(j,i-j))){dp[i]=true;break;}
return dp[s.Length];
}
```

■ O(n²) | ■ O(n)

Q73 String Compression

■ Medium [Two Pointers]

Compress string: 'aaabbc' → 'a3b2c'. Modify in-place, return new length.

Count consecutive same chars, write char + count (omit 1) to write pointer.

```
public int Compress(char[] chars) {
int w=0,i=0;
while(i<chars.Length){
char c=chars[i]; int cnt=0;
while(i<chars.Length&&chars[i]==c){i++;cnt++;}
chars[w++]=c;
if(cnt>1)foreach(char d in cnt.ToString())chars[w++]=d;
}
return w;
}
```

■ O(n) | ■ O(1)

Q74 Multiply Strings

■ Medium [String Math]

Multiply two non-negative integers represented as strings. No big integer library.

Simulate grade-school multiplication: pos[i+j+1] += d1 * d2.

```
public string Multiply(string num1, string num2) {
int m=num1.Length,n=num2.Length;
int[] pos=new int[m+n];
for(int i=m-1;i>=0;i--)for(int j=n-1;j>=0;j--){
int mul=(num1[i]-'0')*(num2[j]-'0');
int p1=i+j,p2=i+j+1,sum=mul+pos[p2];
pos[p2]=sum%10; pos[p1]+=sum/10;
}
var sb=new System.Text.StringBuilder();
foreach(int d in pos) if(!(sb.Length==0&&d==0)) sb.Append(d);
return sb.Length==0?"0":sb.ToString();
}
```

■ O(m·n) | ■ O(m+n)

Q75 Count and Say

■ Medium [String / Simulation]

Generate the nth term of the count-and-say sequence. '1' → '11' → '21' → '1211'...

Repeatedly read current string, count runs of same digit, build next string.

```
public string CountAndSay(int n) {
string s="1";
for(int i=1;i<n;i++){
var sb=new System.Text.StringBuilder(); int j=0;
while(j<s.Length){char c=s[j];int cnt=0;
while(j<s.Length&&s[j]==c){j++;cnt++;}
sb.Append(cnt); sb.Append(c);}
s=sb.ToString();
}
return s;
}
```

■ O(n·2^n) worst | ■ O(2^n)

Chapter 09 — Linked List

Draw the list. Use dummy head nodes to simplify edge cases. Fast/slow pointers for cycle and middle detection.

Q76 Reverse Linked List

■ Easy [Linked List]

Reverse a singly linked list iteratively and recursively.

```
public ListNode ReverseList(ListNode head) {
ListNode prev=null,cur=head;
while(cur!=null){var nxt=cur.next;cur.next=prev;prev=cur;cur=nxt;}
return prev;
}
```

■ O(n) | ■ O(1)

Q77 Detect Cycle in Linked List

■ Easy [Fast/Slow Pointers]

Return true if linked list has a cycle.

Floyd's: slow moves 1, fast moves 2. If they meet, cycle exists.

```
public bool HasCycle(ListNode head) {
var slow=head; var fast=head;
while(fast?.next!=null){slow=slow.next;fast=fast.next.next;
if(slow==fast)return true;}
return false;
}
```

■ O(n) | ■ O(1)

Q78 Find Middle of Linked List

■ Easy [Fast/Slow Pointers]

Return the middle node. For even length, return second middle.

When fast reaches end, slow is at middle.

```
public ListNode MiddleNode(ListNode head) {
var s=head; var f=head;
while(f?.next!=null){s=s.next;f=f.next.next;}
return s;
}
```

■ O(n) | ■ O(1)

Q79 Merge Two Sorted Linked Lists

■ Easy [Linked List]

Merge two sorted linked lists and return sorted result.

Dummy head simplifies edge cases. Compare and append smaller node.

```
public ListNode MergeTwoLists(ListNode l1, ListNode l2) {
var dummy=new ListNode(0); var cur=dummy;
while(l1!=null&&l2!=null){
if(l1.val<=l2.val){cur.next=l1;l1=l1.next;}
else{cur.next=l2;l2=l2.next;}
cur=cur.next;
}
cur.next=l1??l2;
return dummy.next;
}
```

■ O(n+m) | ■ O(1)

Q80 Remove Nth Node From End

■ Medium [Fast/Slow Pointers]

Remove the nth node from the end in one pass.

Advance fast pointer n steps, then move both until fast reaches end.

```
public ListNode RemoveNthFromEnd(ListNode head, int n) {
var dummy=new ListNode(0,head); var fast=dummy; var slow=dummy;
for(int i=0;i<=n;i++) fast=fast.next;
while(fast!=null){slow=slow.next;fast=fast.next;}
slow.next=slow.next.next;
return dummy.next;
}
```

■ O(n) | ■ O(1)

Q81 Add Two Numbers

■ Medium [Linked List Math]

Two numbers stored in reverse linked lists. Return sum as linked list.

Traverse both, add digit by digit with carry. Handle different lengths.

```
public ListNode AddTwoNumbers(ListNode l1, ListNode l2) {
var dummy=new ListNode(0); var cur=dummy; int carry=0;
while(l1!=null||l2!=null||carry!=0){
int s=(l1?.val??0)+(l2?.val??0)+carry;
carry=s/10; cur.next=new ListNode(s%10);
cur=cur.next; l1=l1?.next; l2=l2?.next;
}
return dummy.next;
}
```

■ O(max(m,n)) | ■ O(max(m,n))

Q82 Reorder List

■ Medium [Linked List]

Reorder L0→L1→...→Ln to L0→Ln→L1→Ln-1→...

1) Find middle, 2) Reverse second half, 3) Merge two halves.

```
public void ReorderList(ListNode head) {
// Find middle
var s=head;var f=head;
while(f.next?.next!=null){s=s.next;f=f.next.next;}
// Reverse second half
var prev=new ListNode(0);var cur=s.next; s.next=null;
while(cur!=null){var nxt=cur.next;cur.next=prev.next;prev.next=cur;cur=nxt;}
// Merge
var a=head;var b=prev.next;
while(b!=null){var an=a.next;var bn=b.next;
a.next=b;b.next=an;a=an;b=bn;}
}
```

■ O(n) | ■ O(1)

Q83 LRU Cache

■ Medium [LinkedList + HashMap]

Design LRU cache with O(1) get and put.

Doubly linked list for order + dictionary for O(1) access.

```
public class LRUCache {
int cap; LinkedList<(int k,int v)> list=new();
Dictionary<int,LinkedListNode<(int,int)>> map=new();
public LRUCache(int c)=>cap=c;
public int Get(int k){if(!map.ContainsKey(k))return -1;
var node=map[k]; list.Remove(node); list.AddFirst(node);
return node.Value.v;}
public void Put(int k,int v){if(map.ContainsKey(k))list.Remove(map[k]);
else if(list.Count==cap){map.Remove(list.Last.Value.k);list.RemoveLast();}
list.AddFirst((k,v)); map[k]=list.First;}
}
```

■ O(1) get/put | ■ O(capacity)

Chapter 10 — Trees

Most tree problems use DFS (recursive) or BFS (iterative with Queue). Always handle null base case first.

Q84 Maximum Depth of Binary Tree

■ Easy [DFS]

Return the maximum depth (height) of a binary tree.

```
public int MaxDepth(TreeNode root) {
if(root==null) return 0;
return 1+Math.Max(MaxDepth(root.left),MaxDepth(root.right));
}
```

■ O(n) | ■ O(h)

Q85 Invert Binary Tree

■ Easy [DFS]

Mirror/invert a binary tree.

```
public TreeNode InvertTree(TreeNode root) {
if(root==null) return null;
(root.left,root.right)=(InvertTree(root.right),InvertTree(root.left));
return root;
}
```

■ O(n) | ■ O(h)

Q86 Level Order Traversal

■ Medium [BFS]

Return level-by-level values of a binary tree.

Use Queue. Process all nodes at each level before moving on.

```
public IList<IList<int>> LevelOrder(TreeNode root) {
var res=new List<IList<int>>();
if(root==null) return res;
var q=new Queue<TreeNode>(); q.Enqueue(root);
while(q.Count>0){int sz=q.Count;var lvl=new List<int>();
for(int i=0;i<sz;i++){var n=q.Dequeue();lvl.Add(n.val);
if(n.left!=null)q.Enqueue(n.left);
if(n.right!=null)q.Enqueue(n.right);}
res.Add(lvl);}
return res;
}
```

■ O(n) | ■ O(n)

Q87 Validate Binary Search Tree

■ Medium [DFS with Bounds]

Verify that a binary tree satisfies BST property.

Pass min/max bounds down. Each node must be strictly within (min, max).

```
public bool IsValidBST(TreeNode root) {
bool Check(TreeNode n, long mn, long mx) {
if(n==null) return true;
if(n.val<=mn||n.val>=mx) return false;
return Check(n.left,mn,n.val)&&Check(n.right,n.val,mx);
}
return Check(root,long.MinValue,long.MaxValue);
}
```

■ O(n) | ■ O(h)

Q88 Lowest Common Ancestor of BST

■ Easy [BST Property]

Find LCA of two nodes in a BST.

Both smaller → go left. Both larger → go right. Otherwise current is LCA.

```
public TreeNode LowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
if(p.val<root.val&&q.val<root.val) return LowestCommonAncestor(root.left,p,q);
if(p.val>root.val&&q.val>root.val) return LowestCommonAncestor(root.right,p,q);
return root;
}
```

■ O(h) | ■ O(h)

Q89 Symmetric Tree

■ Easy [DFS]

Check if a binary tree is a mirror of itself.

Compare outer-left with outer-right, inner-left with inner-right recursively.

```
public bool IsSymmetric(TreeNode root) {
bool Mirror(TreeNode l, TreeNode r) {
if(l==null&&r==null) return true;
if(l==null||r==null) return false;
return l.val==r.val&&Mirror(l.left,r.right)&&Mirror(l.right,r.left);
}
return Mirror(root?.left,root?.right);
}
```

■ O(n) | ■ O(h)

Q90 Path Sum

■ Easy [DFS]

Return true if root-to-leaf path with given sum exists.

```
public bool HasPathSum(TreeNode root, int sum) {
if(root==null) return false;
if(root.left==null&&root.right==null) return root.val==sum;
return HasPathSum(root.left,sum-root.val)||
HasPathSum(root.right,sum-root.val);
}
```

■ O(n) | ■ O(h)

Q91 Serialize and Deserialize Binary Tree

■ Hard [BFS / DFS]

Design algorithm to serialize tree to string and deserialize back.

Level order with null markers. Use Queue for deserialise.

```
public string Serialize(TreeNode root) {
var q=new Queue<TreeNode>(); q.Enqueue(root);
var sb=new System.Text.StringBuilder();
while(q.Count>0){var n=q.Dequeue();
if(n==null){sb.Append("null,");continue;}
sb.Append(n.val+","); q.Enqueue(n.left); q.Enqueue(n.right);}
return sb.ToString();
}
public TreeNode Deserialize(string data) {
var vals=data.Split(','); int i=0;
if(vals[i]=="null") return null;
var root=new TreeNode(int.Parse(vals[i++]));
var q=new Queue<TreeNode>(); q.Enqueue(root);
while(q.Count>0){var n=q.Dequeue();
if(vals[i]!="null"){n.left=new TreeNode(int.Parse(vals[i]));q.Enqueue(n.left);}i++;
if(vals[i]!="null"){n.right=new TreeNode(int.Parse(vals[i]));q.Enqueue(n.right);}i++;}
return root;
}
```

■ O(n) | ■ O(n)

Chapter 11 — Recursion & Backtracking

Template: choose → recurse → unchoose. Visualise as a decision tree. Prune early.

Q92 Subsets

■ Medium [Backtracking]

Return all possible subsets (power set) of unique integers.

```
public IList<IList<int>> Subsets(int[] nums) {
var res=new List<IList<int>>();
void BT(int start, List<int> cur){
res.Add(new List<int>(cur));
for(int i=start;i<nums.Length;i++){
cur.Add(nums[i]);BT(i+1,cur);cur.RemoveAt(cur.Count-1);}
}
BT(0,new List<int>());return res;
}
```

■ O(n·2^n) | ■ O(n)

Q93 Permutations

■ Medium [Backtracking]

Return all permutations of distinct integers.

Swap current index with each remaining index, recurse, swap back.

```
public IList<IList<int>> Permute(int[] nums) {
var res=new List<IList<int>>();
void BT(int start){if(start==nums.Length){res.Add(nums.ToList());return;}
for(int i=start;i<nums.Length;i++){
(nums[start],nums[i])=(nums[i],nums[start]);
BT(start+1);(nums[start],nums[i])=(nums[i],nums[start]);}}
BT(0);return res;
}
```

■ O(n·n!) | ■ O(n)

Q94 Combination Sum

■ Medium [Backtracking]

Find all unique combinations of candidates that sum to target. Reuse allowed.

```
public IList<IList<int>> CombinationSum(int[] cands, int target) {
var res=new List<IList<int>>();
void BT(int start,List<int> cur,int rem){
if(rem==0){res.Add(new List<int>(cur));return;}
for(int i=start;i<cands.Length;i++){
if(cands[i]>rem)continue;
cur.Add(cands[i]);BT(i,cur,rem-cands[i]);cur.RemoveAt(cur.Count-1);}
}
BT(0,new List<int>(),target);return res;
}
```

■ O(n^(T/M)) | ■ O(T/M)

Q95 Word Search

■ Medium [Backtracking + DFS]

Given 2D board, check if word exists as adjacent (non-reused) cells.

DFS from each matching start cell. Mark visited, backtrack after.

```
public bool Exist(char[][] board, string word) {
int R=board.Length,C=board[0].Length;
bool DFS(int r,int c,int i){
if(i==word.Length)return true;
if(r<0||r>=R||c<0||c>=C||board[r][c]!=word[i])return false;
char tmp=board[r][c]; board[r][c]='#';
bool ok=DFS(r+1,c,i+1)||DFS(r-1,c,i+1)||DFS(r,c+1,i+1)||DFS(r,c-1,i+1);
board[r][c]=tmp; return ok;}
for(int r=0;r<R;r++)for(int c=0;c<C;c++)
if(DFS(r,c,0))return true;
return false;
}
```

■ O(R·C·4^L) | ■ O(L)

Q96 Generate Parentheses

■ Medium [Backtracking]

Generate all combinations of n pairs of valid parentheses.

Add '(' if open < n. Add ')' if close < open.

```
public IList<string> GenerateParenthesis(int n) {
var res=new List<string>();
void BT(string cur,int open,int close){
if(cur.Length==2*n){res.Add(cur);return;}
if(open<n)BT(cur+'(',open+1,close);
if(close<open)BT(cur+')',open,close+1);}
BT("",0,0);return res;
}
```

■ O(4^n/√n) | ■ O(n)

Q97 Letter Combinations of Phone Number

■ Medium [Backtracking]

Return all letter combinations a phone number digit string could represent.

```
public IList<string> LetterCombinations(string digits) {
if(string.IsNullOrEmpty(digits)) return new List<string>();
string[] map={"abc","def","ghi","jkl","mno","pqrs","tuv","wxyz"};
var res=new List<string>();
void BT(int i,System.Text.StringBuilder sb){
if(i==digits.Length){res.Add(sb.ToString());return;}
foreach(char c in map[digits[i]-'2']){sb.Append(c);BT(i+1,sb);sb.Length--;}
}
BT(0,new System.Text.StringBuilder());return res;
}
```

■ O(4^n·n) | ■ O(n)

Q98 N-Queens

■ Hard [Backtracking]

Place n queens on n×n board so none attack each other. Return all solutions.

Track columns, diagonals (/), anti-diagonals (\) in HashSets.

```
public IList<IList<string>> SolveNQueens(int n) {
var res=new List<IList<string>>();
var cols=new HashSet<int>(); var d1=new HashSet<int>(); var d2=new HashSet<int>();
char[][] board=Enumerable.Range(0,n).Select(_=>new string('.',n).ToCharArray()).ToArray();
void BT(int r){if(r==n){res.Add(board.Select(row=>new string(row)).ToList());return;}
for(int c=0;c<n;c++){if(cols.Contains(c)||d1.Contains(r-c)||d2.Contains(r+c))continue;
cols.Add(c);d1.Add(r-c);d2.Add(r+c);board[r][c]='Q';
BT(r+1);
cols.Remove(c);d1.Remove(r-c);d2.Remove(r+c);board[r][c]='.';}
}
BT(0);return res;
}
```

■ O(n!) | ■ O(n)

Q99 Sudoku Solver

■ Hard [Backtracking]

Solve a Sudoku puzzle by filling empty cells.

For each empty cell try 1-9. Check row, col, 3×3 box. Backtrack if stuck.

```
public void SolveSudoku(char[][] b) {
bool Solve(){for(int r=0;r<9;r++)for(int c=0;c<9;c++){
if(b[r][c]!='.')continue;
for(char d='1';d<='9';d++){
if(!Valid(b,r,c,d)){b[r][c]=d;
if(Solve())return true; b[r][c]='.';}}
return false;}return true;}
Solve();
}
bool Valid(char[][] b,int r,int c,char d){
for(int i=0;i<9;i++){if(b[r][i]==d||b[i][c]==d)return false;
if(b[3*(r/3)+i/3][3*(c/3)+i%3]==d)return false;}return true;}
```

■ O(9^m) m=empty cells | ■ O(m)

Chapter 12 — Dynamic Programming

DP = memoised recursion OR bottom-up tabulation. Always define state clearly: dp[i] means '...'

Q100 Climbing Stairs

■ Easy [DP (Fibonacci)]

Climb 1 or 2 steps. How many distinct ways to reach step n?

```
public int ClimbStairs(int n) {
if(n<=2)return n; int a=1,b=2;
for(int i=3;i<=n;i++)(a,b)=(b,a+b);
return b;
}
```

■ O(n) | ■ O(1)

Q101 Coin Change

■ Medium [DP (Unbounded Knapsack)]

Fewest coins to make amount. Return -1 if impossible.

dp[i] = min coins for amount i. Try every coin.

```
public int CoinChange(int[] coins, int amount) {
int[] dp=new int[amount+1]; Array.Fill(dp,amount+1); dp[0]=0;
for(int i=1;i<=amount;i++)
foreach(int c in coins) if(c<=i) dp[i]=Math.Min(dp[i],dp[i-c]+1);
return dp[amount]>amount?-1:dp[amount];
}
```

■ O(n·a) | ■ O(a)

Q102 Longest Increasing Subsequence

■ Medium [DP]

Length of the longest strictly increasing subsequence.

dp[i] = LIS ending at index i. Transition: dp[i]=max(dp[j]+1) for j less than i where nums[j] less than nums[i].

```
public int LengthOfLIS(int[] nums) {
int n=nums.Length; int[] dp=new int[n]; Array.Fill(dp,1);
int res=1;
for(int i=1;i<n;i++){
for(int j=0;j<i;j++)
if(nums[j]<nums[i]) dp[i]=Math.Max(dp[i],dp[j]+1);
res=Math.Max(res,dp[i]);}
return res;
}
```

■ O(n²) | ■ O(n) | O(n log n) with patience sort (binary search)

Q103 House Robber

■ Medium [DP]

Rob max money without robbing two adjacent houses.

dp[i]=max(dp[i-1], dp[i-2]+nums[i]). Only need last two values.

```
public int Rob(int[] nums) {
int prev2=0,prev1=0;
foreach(int n in nums){int cur=Math.Max(prev1,prev2+n);prev2=prev1;prev1=cur;}
return prev1;
}
```

■ O(n) | ■ O(1)

Q104 0/1 Knapsack

■ Medium [DP (Classic 2D)]

Maximise value with n items, weight limit W. Each item used at most once.

dp[i][w] = max value using first i items with capacity w.

```
public int Knapsack(int W,int[] wt,int[] val,int n) {
int[,] dp=new int[n+1,W+1];
for(int i=1;i<=n;i++) for(int w=0;w<=W;w++){
dp[i,w]=dp[i-1,w];
if(wt[i-1]<=w) dp[i,w]=Math.Max(dp[i,w],dp[i-1,w-wt[i-1]]+val[i-1]);
}
return dp[n,W];
}
```

■ O(n·W) | ■ O(n·W)

Q105 Unique Paths

■ Medium [DP 2D]

Count paths from top-left to bottom-right of m×n grid moving only right or down.

dp[r][c] = dp[r-1][c] + dp[r][c-1]. First row and col are all 1s.

```
public int UniquePaths(int m, int n) {
int[] dp=new int[n]; Array.Fill(dp,1);
for(int r=1;r<m;r++) for(int c=1;c<n;c++) dp[c]+=dp[c-1];
return dp[n-1];
}
```

■ O(m·n) | ■ O(n)

Q106 Edit Distance

■ Hard [DP 2D]

Minimum operations (insert/delete/replace) to convert word1 to word2.

dp[i][j] = edit distance for word1[0..i-1] to word2[0..j-1].

```
public int MinDistance(string w1, string w2) {
int m=w1.Length,n=w2.Length;
int[,] dp=new int[m+1,n+1];
for(int i=0;i<=m;i++) dp[i,0]=i;
for(int j=0;j<=n;j++) dp[0,j]=j;
for(int i=1;i<=m;i++) for(int j=1;j<=n;j++)
dp[i,j]=w1[i-1]==w2[j-1]?dp[i-1,j-1]:
1+Math.Min(dp[i-1,j-1],Math.Min(dp[i-1,j],dp[i,j-1]));
return dp[m,n];
}
```

■ O(m·n) | ■ O(m·n)

Q107 Longest Common Subsequence

■ Medium [DP 2D]

Length of longest subsequence present in both strings.

If chars match, dp[i][j]=dp[i-1][j-1]+1. Else max of skip either.

```
public int LongestCommonSubsequence(string t1, string t2) {
int m=t1.Length,n=t2.Length;
int[,] dp=new int[m+1,n+1];
for(int i=1;i<=m;i++) for(int j=1;j<=n;j++)
dp[i,j]=t1[i-1]==t2[j-1]?dp[i-1,j-1]+1:Math.Max(dp[i-1,j],dp[i,j-1]);
return dp[m,n];
}
```

■ O(m·n) | ■ O(m·n)

Q108 Maximum Product Subarray

■ Medium [DP]

Find maximum product of contiguous subarray.

Track both curMax and curMin (negatives can flip). Compare with global max.

```
public int MaxProduct(int[] nums) {
int mx=nums[0],mn=nums[0],res=nums[0];
for(int i=1;i<nums.Length;i++){
if(nums[i]<0)(mx,mn)=(mn,mx);
mx=Math.Max(nums[i],mx*nums[i]);
mn=Math.Min(nums[i],mn*nums[i]);
res=Math.Max(res,mx);}
return res;
}
```

■ O(n) | ■ O(1)

Q109 Palindrome Partitioning

■ Medium [Backtracking + DP]

Partition string so every substring is a palindrome. Return all partitions.

Backtrack. At each index, try all palindromic prefixes.

```
public IList<IList<string>> Partition(string s) {
var res=new List<IList<string>>();
bool IsPalin(int l,int r){while(l<r)if(s[l++]!=s[r--])return false;return true;}
void BT(int start,List<string> cur){
if(start==s.Length){res.Add(new List<string>(cur));return;}
for(int end=start;end<s.Length;end++)
if(IsPalin(start,end)){cur.Add(s.Substring(start,end-start+1));
BT(end+1,cur);cur.RemoveAt(cur.Count-1);}}
BT(0,new List<string>());return res;
}
```

■ O(n·2^n) | ■ O(n)

Chapter 13 — Greedy, Math & Bit Manipulation

Greedy: make locally optimal choice at each step. Bit tricks replace division/modulo with shift/AND.

Q110 Jump Game

■ Medium [Greedy]

Return true if you can reach the last index from the first.

Track max reachable index. If current index exceeds it, return false.

```
public bool CanJump(int[] nums) {
int reach=0;
for(int i=0;i<nums.Length;i++){
if(i>reach)return false;
reach=Math.Max(reach,i+nums[i]);}
return true;
}
```

■ O(n) | ■ O(1)

Q111 Gas Station

■ Medium [Greedy]

Find starting gas station index to complete circular route. Return -1 if impossible.

If total gas >= total cost, solution exists. Track running surplus; reset start on deficit.

```
public int CanCompleteCircuit(int[] gas, int[] cost) {
int total=0,tank=0,start=0;
for(int i=0;i<gas.Length;i++){
total+=gas[i]-cost[i]; tank+=gas[i]-cost[i];
if(tank<0){start=i+1;tank=0;}}
return total>=0?start:-1;
}
```

■ O(n) | ■ O(1)

Q112 Task Scheduler

■ Medium [Greedy]

Given tasks and cooldown n, find min time to execute all tasks.

Most frequent task determines idle time. Formula: max((maxCount-1)*(n+1)+numMaxTasks, tasks.Length).

```
public int LeastInterval(char[] tasks, int n) {
var cnt=new int[26];
foreach(char t in tasks) cnt[t-'A']++;
int maxCount=cnt.Max();
int numMax=cnt.Count(x=>x==maxCount);
return Math.Max((maxCount-1)*(n+1)+numMax,tasks.Length);
}
```

■ O(n) | ■ O(1)

Q113 Single Number

■ Easy [Bit Manipulation / XOR]

Every element appears twice except one. Find the single one.

XOR of same numbers = 0. XOR of all elements leaves the single number.

```
public int SingleNumber(int[] nums) => nums.Aggregate(0,(a,b)=>a^b);
```

■ O(n) | ■ O(1)

Q114 Number of 1 Bits (Hamming Weight)

■ Easy [Bit Manipulation]

Return number of set bits in 32-bit unsigned integer.

n & (n-1) clears the lowest set bit. Count iterations.

```
public int HammingWeight(uint n) {
int cnt=0; while(n!=0){n&=n-1;cnt++;} return cnt;
}
```

■ O(k) k=set bits | ■ O(1)

Q115 Reverse Bits

■ Easy [Bit Manipulation]

Reverse bits of a 32-bit unsigned integer.

```
public uint ReverseBits(uint n) {
uint res=0;
for(int i=0;i<32;i++){res=(res<<1)|(n&1);n>>=1;}
return res;
}
```

■ O(1) | ■ O(1)

Q116 Power of Three

■ Easy [Math]

Return true if n is a power of three.

Iteratively divide by 3. Or: 3^19=1162261467 is max int power — check divisibility.

```
public bool IsPowerOfThree(int n) {
if(n<=0) return false;
return 1162261467 % n == 0;
}
```

■ O(1) | ■ O(1)

Q117 Excel Sheet Column Number

■ Easy [Math]

Convert Excel column title (e.g. 'AB') to its column number.

Base-26 conversion. 'A'=1, 'B'=2, ... 'Z'=26.

```
public int TitleToNumber(string ct) {
int res=0;
foreach(char c in ct) res=res*26+(c-'A'+1);
return res;
}
```

■ O(n) | ■ O(1)

Q118 Jump Game II

■ Medium [Greedy]

Find minimum number of jumps to reach last index.

Track current range end and next farthest. Increment jumps when end reached.

```
public int Jump(int[] nums) {
int jumps=0,curEnd=0,farthest=0;
for(int i=0;i<nums.Length-1;i++){
farthest=Math.Max(farthest,i+nums[i]);
if(i==curEnd){jumps++;curEnd=farthest;}
}
return jumps;
}
```

■ O(n) | ■ O(1)

Q119 Meeting Rooms II

■ Medium [Greedy + Heap]

Find minimum number of meeting rooms required.

Sort by start. Use min-heap of end times. If earliest ending < new start, reuse.

```
public int MinMeetingRooms(int[][] intervals) {
Array.Sort(intervals,(a,b)=>a[0]-b[0]);
var heap=new PriorityQueue<int,int>();
foreach(var iv in intervals){
if(heap.Count>0&&heap.Peek()<=iv[0]) heap.Dequeue();
heap.Enqueue(iv[1],iv[1]);
}
return heap.Count;
}
```

■ O(n log n) | ■ O(n)

Q120 Find All Numbers Disappeared in Array

■ Easy [Hashing / Index]

Find numbers in range [1,n] missing from array of length n.

Mark visited indices negative. Collect indices still positive.

```
public IList<int> FindDisappearedNumbers(int[] nums) {
foreach(int n in nums){int idx=Math.Abs(n)-1;
if(nums[idx]>0) nums[idx]=-nums[idx];}
var res=new List<int>();
for(int i=0;i<nums.Length;i++) if(nums[i]>0) res.Add(i+1);
return res;
}
```

■ O(n) | ■ O(1)

Chapter 14 — Sorting Algorithms

Knowing sorting internals is essential. Interviewers test trade-offs: stability, in-place, best/worst cases.

Q121 Bubble Sort

■ Easy [Sorting]

Sort array by repeatedly swapping adjacent elements if they are in the wrong order.

Optimization: if no swap in a pass, array is already sorted.

```
public void BubbleSort(int[] arr) {
int n=arr.Length;
for(int i=0;i<n-1;i++){bool swapped=false;
for(int j=0;j<n-i-1;j++)
if(arr[j]>arr[j+1]){(arr[j],arr[j+1])=(arr[j+1],arr[j]);swapped=true;}
if(!swapped)break;}
}
```

■ O(n²) worst/avg, O(n) best | ■ O(1) | Stable ✓

Q122 Selection Sort

■ Easy [Sorting]

Find the minimum of unsorted portion, swap with first unsorted element.

```
public void SelectionSort(int[] arr) {
for(int i=0;i<arr.Length-1;i++){
int minIdx=i;
for(int j=i+1;j<arr.Length;j++)
if(arr[j]<arr[minIdx]) minIdx=j;
(arr[i],arr[minIdx])=(arr[minIdx],arr[i]);
}
}
```

■ O(n²) all cases | ■ O(1) | Unstable ✗

Q123 Insertion Sort

■ Easy [Sorting]

Build sorted array one element at a time by inserting in correct position.

Best for nearly sorted arrays. Shifts rather than swaps.

```
public void InsertionSort(int[] arr) {
for(int i=1;i<arr.Length;i++){
int key=arr[i],j=i-1;
while(j>=0&&arr[j]>key){arr[j+1]=arr[j];j--;}
arr[j+1]=key;
}
}
```

■ O(n²) worst, O(n) best | ■ O(1) | Stable ✓

Q124 Merge Sort

■ Medium [Sorting / Divide & Conquer]

Divide array in half, sort each, merge. Guaranteed O(n log n).

Stable sort. Extra O(n) space for merging.

```
public void MergeSort(int[] arr, int l, int r) {
if(l>=r) return;
int mid=(l+r)/2;
MergeSort(arr,l,mid); MergeSort(arr,mid+1,r);
Merge(arr,l,mid,r);
}
void Merge(int[] arr, int l, int m, int r) {
int[] tmp=arr[l..(r+1)]; int i=0,j=m-l+1,k=l;
while(i<=m-l&&j<=r-l) arr[k++]=tmp[i]<=tmp[j]?tmp[i++]:tmp[j++];
while(i<=m-l) arr[k++]=tmp[i++];
while(j<=r-l) arr[k++]=tmp[j++];
}
```

■ O(n log n) all | ■ O(n) | Stable ✓

Q125 Quick Sort

■ Medium [Sorting / Divide & Conquer]

Partition around pivot, recursively sort sub-arrays.

Lomuto or Hoare partition. Randomise pivot to avoid O(n²) worst case.

```
public void QuickSort(int[] arr, int l, int r) {
if(l>=r) return;
int p=Partition(arr,l,r);
QuickSort(arr,l,p-1); QuickSort(arr,p+1,r);
}
int Partition(int[] arr, int l, int r) {
int pivot=arr[r],i=l-1;
for(int j=l;j<r;j++)
if(arr[j]<=pivot)(arr[++i],arr[j])=(arr[j],arr[i]);
(arr[i+1],arr[r])=(arr[r],arr[i+1]);
return i+1;
}
```

■ O(n log n) avg, O(n²) worst | ■ O(log n) | Unstable ✗

Chapter 14 (continued) — Additional Problems

Q126 Counting Sort

■ Easy [Sorting / Non-Comparison]

Sort integers in known range [0..k] in O(n+k).

```
public void CountingSort(int[] arr, int k) {
int[] cnt=new int[k+1];
foreach(int n in arr) cnt[n]++;
int idx=0;
for(int i=0;i<=k;i++) while(cnt[i]-->0) arr[idx++]=i;
}
```

■ O(n+k) | ■ O(k) | Stable ✓

Q127 Kth Largest Element

■ Medium [Quickselect / Heap]

Find kth largest element in unsorted array.

Min-heap of size k. Or Quickselect for O(n) average.

```
public int FindKthLargest(int[] nums, int k) {
var pq=new PriorityQueue<int,int>();
foreach(int n in nums){pq.Enqueue(n,n);if(pq.Count>k)pq.Dequeue();}
return pq.Peek();
}
```

■ O(n log k) | ■ O(k)

Q128 Find Peak Element

■ Medium [Binary Search]

A peak element is greater than its neighbours. Find any peak index.

Binary search: if nums[mid] < nums[mid+1], peak is on right; else left.

```
public int FindPeakElement(int[] nums) {
int lo=0,hi=nums.Length-1;
while(lo<hi){int mid=lo+(hi-lo)/2;
if(nums[mid]<nums[mid+1])lo=mid+1; else hi=mid;}
return lo;
}
```

■ O(log n) | ■ O(1)

Q129 Minimum Rotations to Sort

■ Medium [Greedy]

Find the number of rotations to sort a rotated sorted array. (Find rotation pivot index.)

Count 'drops' where nums[i] > nums[i+1]. The number of drops = rotation count for a rotated array.

```
public int MinRotations(int[] nums) {
int drops=0, pivot=0;
for(int i=0;i<nums.Length-1;i++)
if(nums[i]>nums[i+1]){drops++;pivot=i+1;}
return drops==1?pivot:0; // 0 if already sorted, -1 if invalid
}
```

■ O(n) | ■ O(1)

Q130 Interval Intersection

■ Medium [Two Pointers]

Find intersection of two sorted interval lists.

Use two pointers. Intersect if overlap exists. Advance pointer with earlier end.

```
public int[][] IntervalIntersection(int[][] A, int[][] B) {
var res=new List<int[]>(); int i=0,j=0;
while(i<A.Length&&j<B.Length){
int lo=Math.Max(A[i][0],B[j][0]),hi=Math.Min(A[i][1],B[j][1]);
if(lo<=hi) res.Add(new[]{lo,hi});
if(A[i][1]<B[j][1])i++; else j++;
}
return res.ToArray();
}
```

■ O(n+m) | ■ O(n+m)

Chapter 14 (Extra) — Mixed Hard Problems

Q131 Longest Substring with At Most K Distinct Chars ■ Medium [Sliding Window]

Find longest substring containing at most k distinct characters.

Sliding window with char frequency map. Shrink when distinct > k.

```
public int LengthOfLongestSubstringKDistinct(string s, int k) {
var map=new Dictionary<char,int>(); int l=0,res=0;
for(int r=0;r<s.Length;r++){
map[s[r]]=map.GetValueOrDefault(s[r],0)+1;
while(map.Count>k){map[s[l]]--;if(map[s[l]]==0)map.Remove(s[l]);l++;}
res=Math.Max(res,r-l+1);}
return res;
}
```

■ O(n) | ■ O(k)

Q132 Ransom Note ■ Easy [Hashing]

Return true if ransom note can be built using letters from magazine.

```
public bool CanConstruct(string ransom, string magazine) {
var cnt=new int[26];
foreach(char c in magazine) cnt[c-'a']++;
foreach(char c in ransom){cnt[c-'a']--;if(cnt[c-'a']<0)return false;}
return true;
}
```

■ O(n) | ■ O(1)

Q133 Number of Islands ■ Medium [BFS/DFS]

Count number of islands in 2D grid ('1'=land, '0'=water).

DFS from each unvisited land cell, mark visited as '0'.

```
public int NumIslands(char[][] g) {
int cnt=0,R=g.Length,C=g[0].Length;
void DFS(int r,int c){
if(r<0||r>=R||c<0||c>=C||g[r][c]!='1')return;
g[r][c]='0'; DFS(r+1,c);DFS(r-1,c);DFS(r,c+1);DFS(r,c-1);}
for(int r=0;r<R;r++) for(int c=0;c<C;c++)
if(g[r][c]=='1'){DFS(r,c);cnt++;}
return cnt;
}
```

■ O(R·C) | ■ O(R·C)

Q134 Clone Graph

■ Medium [BFS + HashMap]

Clone an undirected graph. Each node has val and list of neighbors.

HashMap of original→clone. BFS to visit all nodes.

```
public Node CloneGraph(Node node) {
if(node==null) return null;
var map=new Dictionary<Node,Node>();
var q=new Queue<Node>(); q.Enqueue(node);
map[node]=new Node(node.val);
while(q.Count>0){var n=q.Dequeue();
foreach(var nb in n.neighbors){
if(!map.ContainsKey(nb)){map[nb]=new Node(nb.val);q.Enqueue(nb);}
map[n].neighbors.Add(map[nb]);}}
return map[node];
}
```

■ O(V+E) | ■ O(V)

Q135 Course Schedule (Cycle Detection)

■ Medium [Topological Sort / DFS]

Determine if all courses can be finished given prerequisites (detect cycle in DAG).

Build adjacency list. DFS with state: 0=unvisited, 1=visiting, 2=visited.

```
public bool CanFinish(int n, int[][] prereqs) {
var adj=new List<int>[n]; for(int i=0;i<n;i++) adj[i]=new List<int>();
foreach(var p in prereqs) adj[p[1]].Add(p[0]);
int[] state=new int[n];
bool DFS(int v){if(state[v]==1)return false;if(state[v]==2)return true;
state[v]=1;foreach(int nb in adj[v])if(!DFS(nb))return false;
state[v]=2;return true;}
for(int i=0;i<n;i++) if(!DFS(i)) return false;
return true;
}
```

■ O(V+E) | ■ O(V+E)

Q136 Max Points on a Line

■ Hard [Hashing + Math]

Find max number of points that lie on the same straight line.

For each point, compute slope to every other. Same slope = same line. Use GCD to normalize.

```
public int MaxPoints(int[][] pts) {
int n=pts.Length,res=1;
for(int i=0;i<n;i++){var map=new Dictionary<string,int>();
for(int j=i+1;j<n;j++){
int dy=pts[j][1]-pts[i][1],dx=pts[j][0]-pts[i][0];
int g=GCD(Math.Abs(dy),Math.Abs(dx));
if(g!=0){dy/=g;dx/=g;} if(dx<0){dx=-dx;dy=-dy;}
string key=$"{dy}/{dx}";
map[key]=map.GetValueOrDefault(key,1)+1;
res=Math.Max(res,map[key]);}
}
return res;
}
int GCD(int a,int b)=>b==0?a:GCD(b,a%b);
```

■ O(n²) | ■ O(n)

Q137 Spiral Matrix

■ Medium [Simulation]

Return elements of m×n matrix in spiral order.

Four boundaries: top, bottom, left, right. Shrink after each direction pass.

```
public IList<int> SpiralOrder(int[][] matrix) {
var res=new List<int>();
int top=0,bot=matrix.Length-1,l=0,r=matrix[0].Length-1;
while(top<=bot&&l<=r){
for(int c=l;c<=r;c++) res.Add(matrix[top][c]); top++;
for(int row=top;row<=bot;row++) res.Add(matrix[row][r]); r--;
if(top<=bot){for(int c=r;c>=l;c--) res.Add(matrix[bot][c]); bot--;}
if(l<=r){for(int row=bot;row>=top;row--) res.Add(matrix[row][l]); l++;}
}
return res;
}
```

■ O(m·n) | ■ O(1)

Q138 Rotate Image 90° ■ Medium [Matrix]

Rotate n×n matrix 90 degrees clockwise in-place.

Transpose (swap [i][j] with [j][i]), then reverse each row.

```
public void Rotate(int[][] m) {
int n=m.Length;
// Transpose
for(int i=0;i<n;i++) for(int j=i+1;j<n;j++)
(m[i][j],m[j][i])=(m[j][i],m[i][j]);
// Reverse rows
foreach(var row in m) Array.Reverse(row);
}
```

■ O(n²) | ■ O(1)

Q139 Set Matrix Zeroes ■ Medium [Array]

If element is 0, set its entire row and column to 0. Do in-place.

Use first row/col as markers. Process body, then handle first row/col separately.

```
public void SetZeroes(int[][] m) {
int R=m.Length,C=m[0].Length;
bool fr=m[0].Any(x=>x==0),fc=false;
for(int r=0;r<R;r++) if(m[r][0]==0) fc=true;
for(int r=1;r<R;r++) for(int c=1;c<C;c++)
if(m[r][c]==0){m[r][0]=0;m[0][c]=0;}
for(int r=1;r<R;r++) for(int c=1;c<C;c++)
if(m[r][0]==0||m[0][c]==0) m[r][c]=0;
if(fr) for(int c=0;c<C;c++) m[0][c]=0;
if(fc) for(int r=0;r<R;r++) m[r][0]=0;
}
```

■ O(m·n) | ■ O(1)

Q140 Minimum Path Sum in Grid

■ Medium [DP 2D]

Find path from top-left to bottom-right minimising sum. Move only right or down.

dp[r][c] = min cost to reach cell. dp[r][c] += min(dp[r-1][c], dp[r][c-1]).

```
public int MinPathSum(int[][] grid) {
int R=grid.Length,C=grid[0].Length;
for(int r=0;r<R;r++) for(int c=0;c<C;c++){
if(r==0&&c==0) continue;
int up=r>0?grid[r-1][c]:int.MaxValue;
int lt=c>0?grid[r][c-1]:int.MaxValue;
grid[r][c]+=Math.Min(up,lt);}
return grid[R-1][C-1];
}
```

■ O(m·n) | ■ O(1)

Q141 Decode Ways

■ Medium [DP]

Count ways to decode digit string where A=1,...,Z=26.

dp[i] = ways to decode s[0..i-1]. Single digit or two-digit decode transitions.

```
public int NumDecodings(string s) {
int n=s.Length; int[] dp=new int[n+1]; dp[0]=1;
dp[1]=s[0]=='0'?0:1;
for(int i=2;i<=n;i++){
if(s[i-1]!='0') dp[i]+=dp[i-1];
int two=int.Parse(s.Substring(i-2,2));
if(two>=10&&two<=26) dp[i]+=dp[i-2];
}
return dp[n];
}
```

■ O(n) | ■ O(n)

Q142 Word Ladder

■ Hard [BFS]

Minimum word transformations from beginWord to endWord, changing one letter at a time.

BFS from begin. For each word, try all 26 letter substitutions. Track visited.

```
public int LadderLength(string begin, string end, IList<string> wordList) {
var wordSet=new HashSet<string>(wordList);
if(!wordSet.Contains(end)) return 0;
var q=new Queue<string>(); q.Enqueue(begin); int steps=1;
while(q.Count>0){int sz=q.Count;
for(int i=0;i<sz;i++){var w=q.Dequeue().ToCharArray();
for(int j=0;j<w.Length;j++){char orig=w[j];
for(char c='a';c<='z';c++){w[j]=c;string nw=new string(w);
if(nw==end)return steps+1;
if(wordSet.Remove(nw))q.Enqueue(nw);}w[j]=orig;}}
steps++;}
return 0;
}
```

■ O(n·m·26) | ■ O(n)

Q143 Trie - Implement Insert/Search/StartsWith

■ Medium [Trie]

Implement a Trie (prefix tree) with insert, search, and startsWith operations.

```
public class Trie {
Trie[] ch=new Trie[26]; bool end;
public void Insert(string w){var t=this;
foreach(char c in w){int i=c-'a';t.ch[i]??=new Trie();t=t.ch[i];}
t.end=true;}
public bool Search(string w){var t=this;
foreach(char c in w){int i=c-'a';if(t.ch[i]==null)return false;t=t.ch[i];}
return t.end;}
public bool StartsWith(string p){var t=this;
foreach(char c in p){int i=c-'a';if(t.ch[i]==null)return false;t=t.ch[i];}
return true;}
}
```

■ O(m) all ops | ■ O(ALPHABET·n·m)

Q144 Design HashMap

■ Easy [Hashing / Design]

Design a HashMap without built-in hash table libraries.

Use array of buckets (linked lists) to handle collisions.

```
public class MyHashMap {
const int SIZE=1000;
List<(int k,int v)>[] buckets=new List<(int,int)>[SIZE];
int Idx(int k)=>k%SIZE;
public void Put(int k,int v){int i=Idx(k);buckets[i]??=new();
var b=buckets[i]; for(int j=0;j<b.Count;j++)
if(b[j].k==k){b[j]=(k,v);return;} b.Add((k,v));}
public int Get(int k){int i=Idx(k); if(buckets[i]==null) return -1;
return buckets[i].FirstOrDefault(p=>p.k==k,(-1,-1)).v;}
public void Remove(int k){int i=Idx(k);
buckets[i]?.RemoveAll(p=>p.k==k);}
}
```

■ O(n/k) avg | ■ O(n)

Q145 Flatten Nested List Iterator

■ Medium [Stack / Design]

Implement iterator that flattens a nested list of integers.

Pre-flatten using recursive DFS, or use stack with lazy evaluation.

```
// Pre-flatten approach
public class NestedIterator {
Queue<int> q=new();
public NestedIterator(IList<NestedInteger> nl){
void Flatten(IList<NestedInteger> list){
foreach(var ni in list)
if(ni.IsInteger())q.Enqueue(ni.GetInteger());
else Flatten(ni.GetList());}
Flatten(nl);}
public int Next()=>q.Dequeue();
public bool HasNext()=>q.Count>0;
}
```

■ O(n) init | ■ O(n)

Chapter 14 — Final Five Challenges

Q146 Regular Expression Matching

■ Hard [DP 2D]

Implement regex matching with '.' (any char) and '*' (zero or more of preceding).

dp[i][j] = s[0..i-1] matches p[0..j-1]. Handle '*' with two choices: zero-use or one-use.

```
public bool IsMatch(string s, string p) {
int m=s.Length,n=p.Length;
bool[,] dp=new bool[m+1,n+1]; dp[0,0]=true;
for(int j=1;j<=n;j++)
if(p[j-1]=='*') dp[0,j]=j>=2&&dp[0,j-2];
for(int i=1;i<=m;i++) for(int j=1;j<=n;j++){
if(p[j-1]=='*')
dp[i,j]=dp[i,j-2]||(j>=2&&dp[i-1,j]&&(p[j-2]=='.'||p[j-2]==s[i-1]));
else dp[i,j]=dp[i-1,j-1]&&(p[j-1]=='.'||p[j-1]==s[i-1]);}
return dp[m,n];
}
```

■ O(m·n) | ■ O(m·n)

Q147 Burst Balloons

■ Hard [Interval DP]

Burst all balloons to maximise coins. Coins = left*curr*right neighbours.

dp[l][r] = max coins bursting all balloons between l and r. Try each as last burst.

```
public int MaxCoins(int[] nums) {
var a=new[]{1}.Concat(nums).Concat(new[]{1}).ToArray();
int n=a.Length; int[,] dp=new int[n,n];
for(int len=2;len<n;len++)
for(int l=0;l<n-len;l++){int r=l+len;
for(int k=l+1;k<r;k++)
dp[l,r]=Math.Max(dp[l,r],dp[l,k]+a[l]*a[k]*a[r]+dp[k,r]);}
return dp[0,n-1];
}
```

■ $O(n^3)$ | ■ $O(n^2)$

Q148 Alien Dictionary

■ Hard [Topological Sort]

Given sorted alien language word list, find character ordering.

Compare adjacent words to extract char ordering edges. Topo sort with cycle check.

```
// Simplified BFS topo sort
public string AlienOrder(string[] words) {
var adj=new Dictionary<char,HashSet<char>>();
var inDeg=new Dictionary<char,int>();
foreach(char c in string.Concat(words)){adj.TryAdd(c,new());inDeg.TryAdd(c,0);}
for(int i=0;i<words.Length-1;i++){string a=words[i],b=words[i+1];
for(int j=0;j<Math.Min(a.Length,b.Length);j++){
if(a[j]!=b[j]){if(adj[a[j]].Add(b[j]))inDeg[b[j]]++;break;}
if(j==b.Length-1&&a.Length>b.Length) return "";}
}
var q=new Queue<char>(inDeg.Where(p=>p.Value==0).Select(p=>p.Key));
var sb=new System.Text.StringBuilder();
while(q.Count>0){char c=q.Dequeue();sb.Append(c);
foreach(char nb in adj[c]) if(--inDeg[nb]==0) q.Enqueue(nb);}
return sb.Length==inDeg.Count?sb.ToString():"";
}
```

■ O(C) C=total chars | ■ O(1) (26 chars max)

Q149 Maximum Flow (Ford-Fulkerson concept)

■ Hard [Graph]

Find max flow from source to sink in a directed weighted graph.

BFS (Edmonds-Karp) to find augmenting paths. Update residual graph each time.

```
public int MaxFlow(int[,] cap, int s, int t) {
int n=cap.GetLength(0),flow=0;
int[] parent=new int[n];
bool BFS(){Array.Fill(parent,-1);parent[s]=s;
var q=new Queue<int>(); q.Enqueue(s);
while(q.Count>0){int u=q.Dequeue();
for(int v=0;v<n;v++) if(parent[v]==-1&&cap[u,v]>0){
parent[v]=u; if(v==t)return true; q.Enqueue(v);}}
return false;}
while(BFS()){int path=int.MaxValue,v=t;
while(v!=s){path=Math.Min(path,cap[parent[v],v]);v=parent[v];}
v=t; while(v!=s){cap[parent[v],v]-=path;cap[v,parent[v]]+=path;v=parent[v];}
flow+=path;}
return flow;
}
```

■ O(V·E²) | ■ O(V)

Q150 Design Twitter

■ Hard [OOP + Heap]

Design simplified Twitter: postTweet, getNewsFeed (top 10), follow, unfollow.

User→set of followees. Posts stored with global timestamp. Min-heap for top-10 merge.

```
public class Twitter {
int time=0; Dictionary<int,List<(int t,int id)>> tweets=new();
Dictionary<int,HashSet<int>> follows=new();
void EnsureUser(int u){tweets.TryAdd(u,new());follows.TryAdd(u,new HashSet<int>{u});}
public void PostTweet(int u,int id){EnsureUser(u);tweets[u].Add((time++,id));}
public IList<int> GetNewsFeed(int u){EnsureUser(u);
var pq=new PriorityQueue<(int t,int id),int>();
foreach(int f in follows[u]) foreach(var tw in tweets.GetValueOrDefault(f,new()))
pq.Enqueue(tw,-tw.t);
var res=new List<int>();
while(pq.Count>0&&res.Count<10) res.Add(pq.Dequeue().id);
return res;}
public void Follow(int u,int f){EnsureUser(u);follows[u].Add(f);}
public void Unfollow(int u,int f){follows.GetValueOrDefault(u)?.Remove(f);}
}
```

■ Post O(1), Feed O(n log n) | ■ O(n)

Chapter 15 — Quick Reference

Pattern Decision Table

Pattern	Signal Keywords	C# Structures
Two Pointers	sorted, in-place, pair sum, palindrome	int l,r; while(l<r)
Sliding Window	substring, subarray, contiguous, window	HashSet/Dict + l,r pointers
Hashing	O(1) lookup, frequency, anagram, duplicate	Dictionary<K,V>, HashSet<T>
Binary Search	sorted, O(log n), min/max feasibility	lo,hi,mid pattern
Monotonic Stack	next greater/smaller, temperatures, span	Stack<int> (indices)
BFS	shortest path, level order, word ladder	Queue<T>, visited set
DFS/Backtracking	all paths, combinations, permutations	Recursion + undo step
DP 1D	count ways, min cost, linear recurrence	int[] dp or 2 variables
DP 2D	two strings, grid, knapsack	int[,] dp
Greedy	locally optimal, interval, scheduling	Sort + single pass
Prefix Sum	range sum query, subarray sum	int[] prefix
Heap	top-k, kth element, streaming min/max	PriorityQueue<T,P>
Union-Find	connected components, cycle undirected	int[] parent, rank
Trie	prefix search, autocomplete, word dict	class Trie{Trie[] ch}

Complexity Quick Reference

Algorithm	Time	Space	Stable
Bubble Sort	O(n²)	O(1)	Yes
Selection Sort	O(n²)	O(1)	No
Insertion Sort	O(n²)	O(1)	Yes
Merge Sort	O(n log n)	O(n)	Yes
Quick Sort	O(n log n) avg	O(log n)	No
Counting Sort	O(n+k)	O(k)	Yes
Heap Sort	O(n log n)	O(1)	No
Binary Search	O(log n)	O(1)	—
BFS / DFS	O(V+E)	O(V)	—
Dijkstra	O((V+E)logV)	O(V)	—

C# Developer Interview Tips

- Use **var** for type inference, but be explicit in method signatures for readability.
- Prefer **Dictionary.GetValueOrDefault(key, default)** over ContainsKey + indexer (one lookup).
- **Array.Sort()** uses Introsort (O(n log n)); use **LINQ OrderBy()** for stable sort.
- Use **StringBuilder** for all string building in loops — string concatenation is O(n²).
- **(a, b) = (b, a)** — tuple deconstruction for clean in-place swaps.
- **nums[^1]** — C# 8 index-from-end operator for last element.
- **nums[1..^1]** — Range slice operator for elegant subarray slicing.
- Use **PriorityQueue** (.NET 6+) for heap operations.

- **Array.Fill(dp, value)** to initialise DP arrays in one line.
- For large sums, cast to **long** early: **(long)a + b** before the operation.
- LINQ is expressive but O(n) hidden allocations — avoid in tight inner loops.
- Always clarify: null input? Integer overflow? Should input be mutated? 0-indexed or 1-indexed?

You've covered 150 questions. You're ready. Go crush it! ■

— Good luck from your C# DSA Booklet —